

The crew plugin: an overview for everyone

The high-level guide. No code knowledge needed. A companion technical reference covers the same ground in depth. Written 20 September 2026 against crew 0.19.92, with the Stop-gate fix (0.19.93) described as it stands in review.

In one paragraph. Crew turns an AI coding assistant into a small, disciplined software team. One member manages, one investigates, one builds, an independent one from a different AI company reviews, and automatic checks stop anything that fails from being merged or released. Everything the team learns about a system is written down next to the code, so nothing depends on one person's memory or one chat window.

1. The problem

An AI assistant that writes software has four expensive habits when it works alone.

Habit	What it costs	What crew does about it
It marks its own homework.	The same model that wrote a change is the one saying the change is fine.	A reviewer from a different AI company reads every change, looking for defects, before anything merges.
It forgets everything between conversations.	Each session starts from zero. It does not know how the database is laid out or what broke last month.	The team writes a map of the system, kept with the code, and reads it at the start of every session.
It wanders.	Asked to fix one thing, it fixes five, and the one you asked for arrives with strangers attached.	One ticket, one job. Anything else it notices is written down as a follow-up, not done.
Nothing stops it.	A wrong command against a live system, or a release with a failing test, goes through unless a person is watching.	Automatic gates block the merge or the release. Dangerous commands are refused outright.

2. How it works

Think of a very small consultancy inside the editor.

- **You write a ticket.** Crew helps turn "can we make this faster" into a bounded piece of work with a definition of done.
- **A manager decides who does what.** The manager reads the state of the project, hands out the work, keeps the overall picture, and never writes the code itself.
- **A builder does that one job** in a private workspace, then hands the change back with a summary.
- **A reviewer from a different AI company** reads the change and reports defects in a fixed format, or says it is clean.
- **Gates run automatically.** If a check fails, the change does not merge and the release does not happen. Nobody has to remember.
- **Everything is written down:** decisions, lessons, hand-over notes, and the map of the system. The next session, or the next person, starts where this one stopped.

3. Who is on the team

Crew starts small and grows only when there is evidence it should. A role is added when something has gone wrong that the role would have caught; it is not added because it sounds useful.

Member	Job
Manager	Keeps the picture, assigns work, reports status. Since this week it is mechanically prevented from writing code.
Explorer	Answers "where does this live and how does it work". It can read but never change anything.
Builder	Implements one ticket at a time. Never reviews its own work.
Reviewer	Hostile, independent review of every change, from a different AI company where possible.
Security reviewer	Reads anything touching logins, secrets, databases, uploads or cloud permissions before it merges.
Test writer	Builds the quick checks a project runs on every change, so the gates have something to stand on.

Member	Job
Planner, analyst, researcher	A second opinion on a design; a survey of what should improve next; answers from outside sources with references.
Database reviewer, documentation writer, browser tester, scribe, cloud architect	Added as a project grows: migration review, documents generated from code, user-interface tests, the record of decisions, and a check on cloud designs before they are built.

Beyond these, forty specialists can be switched on for a particular project: experts in specific languages, cloud platforms, Microsoft products, security testing, compliance, and writing. They are switched on because a project uses that technology, not because the team has grown.

4. What you can ask for

You want to...	What crew does
Set up a project	A guided, step-by-step setup you can pause and resume. Then it learns the codebase once and writes down what it found.
Get a piece of work done	Turn the request into a ticket, build it, have it reviewed, and run the gates. One ticket in, one reviewed change out.
Know where things stand	Ask the manager. It reports what is open, what is healthy, what is stale, and what it recommends.
Find out why something is broken	A systematic investigation that proves the cause before anyone proposes a fix.
Decide what to improve next	A survey of real gaps with options and their trade-offs.
Check a design before building it	An independent opinion from someone who has not seen the code.
Release to the next environment	A promotion with the evidence attached, and the change-request paperwork filled in from the ticket.
Handle an emergency	A break-glass path that stands the gates down, investigates in parallel, and records exactly what was skipped.
Keep documents honest	The changelog, the reference manual, diagrams and runbooks are updated from the code, not by hand.
Connect your tools	Jira, ServiceDesk Plus or an Obsidian board for tickets; Teams or Telegram for notifications; AWS and Azure with guardrails.

5. Safety

Some checks must not depend on anyone remembering. Crew wires those into the editor so they run by themselves at fixed moments, whether or not the AI agrees.

- **Before a command runs**, dangerous ones are refused: applying infrastructure changes, force-pushing, bypassing a merge protection, or touching anything you have declared as production.
- **Before a file is written** (new this week), each team member may only write where its job allows. The manager cannot quietly become the builder.
- **At the end of every turn**, the checks for the files that changed run, and a failure blocks the turn. This is the safety net under every change, and the subject of section 7.
- **Before a merge or a release**, the same checks must be green, and a release without its evidence or its change request is refused.

Two rules govern all of this. A check that can block ships switched off and is turned on deliberately. And when two settings disagree, or a setting cannot be read, the stricter one wins. Nothing ever quietly becomes "off".

6. What changed this week

Change	What it means for you
Team members no longer stall waiting for a task that will never wake them, and the manager no longer restarts a member that is still working.	Three agents in one night had frozen forever "waiting for a notification". That cannot happen now, and live work is not thrown away.
The rules are now tested against a real AI model, not just written down.	The very first run caught the manager writing code itself, which the rules forbid.
The manager, and every other member, is mechanically limited to the files its job allows.	The rule that the manager must not write code had been broken for four hours in one session. Now the tool refuses. It is switched on per project.
Four command definitions that had silently failed to load were fixed, and a check now catches that class of breakage.	Silent breakage in the team's own definitions is now a failing check.

7. The Stop-gate fix

The problem. The safety net at the end of every turn was taking seven minutes, every turn, and people were cancelling it. A safety net that people switch off protects nobody.

Why. The net remembers the last point it fully checked, and only checks what changed since. But it only moved that point forward when every check had run. One check took longer than the time allowed, so it was always postponed, the point never moved, and every turn re-checked everything since a commit from days ago. Separately, a check with no stated cost could never be postponed at all, so those ran every turn regardless. Two other projects on the same machine were worse: one had never recorded a checkpoint and had no costs stated, and another had ten checks that, on every turn, sent commands to a live development server.

The fix.

- Each check is remembered on its own. One slow check no longer freezes progress for all the others. A postponed check is listed every turn until it is run, so nothing is forgotten.
- Checks measure their own cost the first time they run, so they can be scheduled sensibly afterwards.
- Each check declares whether it stays on this machine or reaches a server. Only local checks run automatically. A check that runs a script but has not declared itself is held until someone declares it, and the manager tells you which ones.
- Leftover settings in a terminal can no longer point a check at a production system.
- A check that could not run because its environment was absent is not counted as passed.

Where it stands. The fix is built and has been through five independent review rounds plus a specialist review, with 31 findings fixed and a test written for each. One of those tests runs this project's own real checks through the new logic and caught three mistakes that would have hurt this project directly. **One last fix is in progress, then one more review round, then it merges.** The quick fix already applied to this machine took the net here from seven minutes to a few seconds.

What a project has to do once it arrives. Declare which of its checks run scripts, price its checks once or let them price themselves, and record its checkpoint only on a commit that has actually been verified.

8. Getting started

1. Install crew in Claude Code.
2. In a project, run the guided setup. It asks before it writes anything.
3. Build the quick-check harness when asked. Everything else stands on it.
4. Let crew learn the codebase once.
5. Write a ticket, let the team work it, read the review, merge when the gate is green.

9. Glossary

Term	Meaning
Agent, role, member	A separately-instructed copy of the AI with its own job, tools and memory.
Hook	A check the editor runs by itself at a fixed moment, whether or not the AI wants it to.
Gate	A check that blocks progress when it fails, rather than giving advice.
Guard	A hook that refuses one specific dangerous action.
Ticket	One bounded piece of work with a definition of done.
Code map	The written description of how a system works, kept with the code.
Stop gate	The safety net that runs at the end of every turn.
Reach	Whether a check stays on this machine, touches the network, or touches a server.

Provenance: written from the repository at commit 621d50dc and the Stop-gate branch at b6e8c4bb. Not verified: the Stop-gate fix's effect on live sessions, pending merge.